



CNFP1 Session 29 - Terraform Session 02 + Vault Session 01[22-Aug-2024]

Status	Done
Tag	Terraform
URL	https://youtu.be/k1EoBR1fh9A?si=TCqV6rZe0Y8zmlZY

Access key, secret access key has lifetime accessment. So, vault can rotate key.

`-dev-listen-address=0.0.0.0:8200`

- **Public / Network-wide.**
Accessible from any device on local network, remote clients, and outside Docker containers.
- Binds to **all available network interfaces** on the host machine.
- Running Vault inside a **Docker container**, VM, or an AWS EC2 instance where external tools need to connect.

`-dev-listen-address=0.0.0.0:8200`

- **Localhost-only.** Only accessible from the exact machine or container where Vault is running.
- Binds strictly to the **loopback interface**.
- Basic **local development** directly on workstation operating system.

```
→ vault server -dev -dev-listen-address=0.0.0.0:8200
```

Need to set environment variables.

```
→ export VAULT_ADDR='http://0.0.0.0:8200'
→ export VAULT_TOKEN=xxxxxxxxxxxxxxxxxxxxxxxx
```

```
→ vault status
```

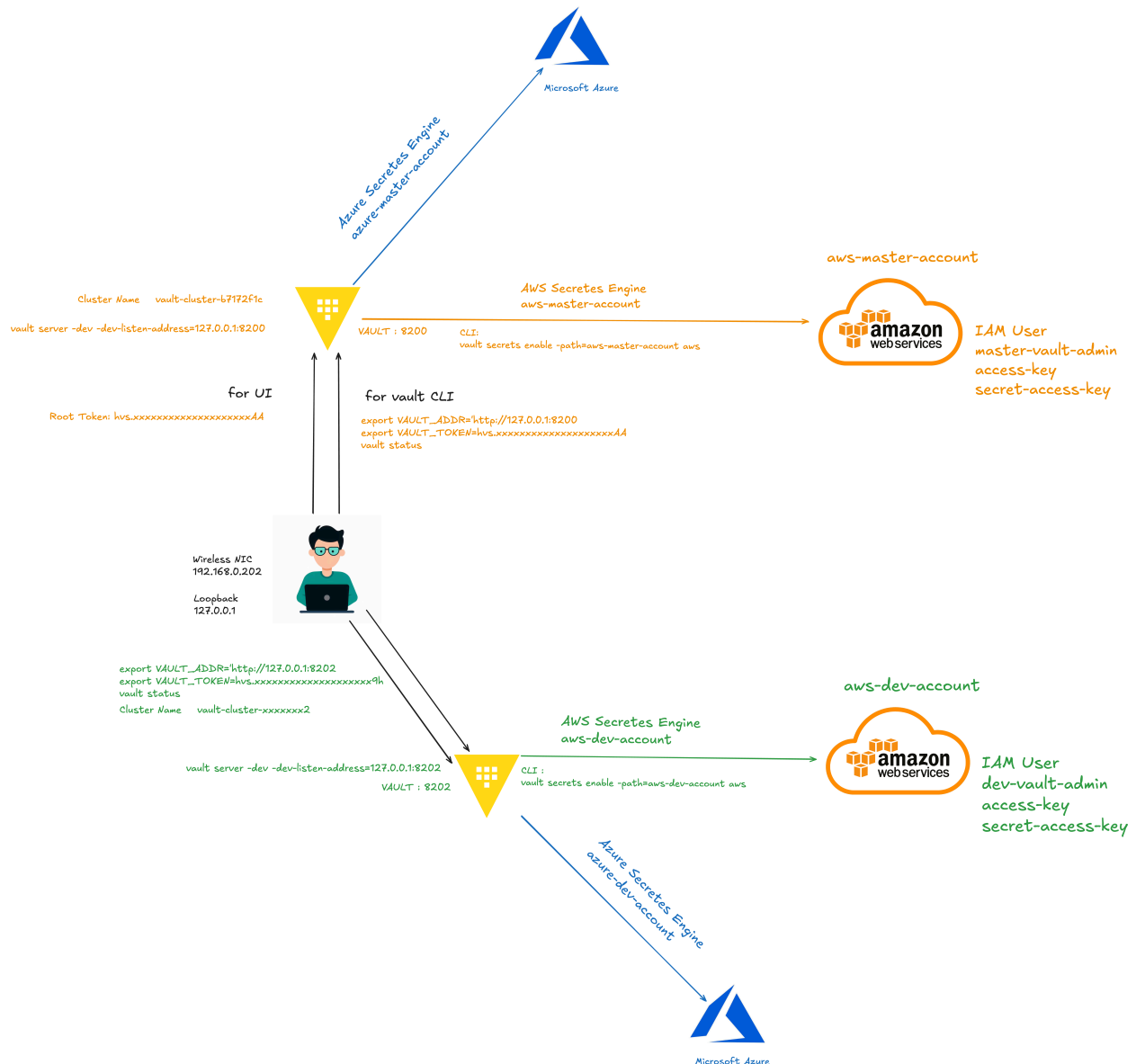
Key	Value
---	----
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	1
Threshold	1
Version	2.0.3
Build Date	2026-06-17T12:39:45Z
Storage Type	inmem
Cluster Name	vault-cluster-xxxxxxx
Cluster ID	370b8618-90b4-3864-78e5-xxxxxxxxxxx
HA Enabled	false

1. Start vault server.
2. Log in with UI or CLI.
3. Configure secret engine to communicate cloud environment (AWS, Azure, ...)

```
→ vault secrets enable -path=aws-master-account aws
```

Create secret engine using CLI.

4. Need to have security mechanism to access API.(IAM User with access key, secret access key in AWS)



▼ Check it out!

terraform/tf-vault-basic-demo

```
→ tf-vault-basic-demo git:(main) x terraform init
Initializing the backend...
```

```
Initializing provider plugins...
```

- Finding hashicorp/vault versions matching "5.11.0"...
- Installing hashicorp/vault v5.11.0...
- Installed hashicorp/vault v5.11.0 (signed by HashiCorp)

Terraform has created a lock **file** `.terraform.lock.hcl` to record the provider selections it made above. Include this **file** in your version control repository so that Terraform can guarantee to **make** the same selections by default when you run `"terraform init"` in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running `"terraform plan"` to see any changes that are required **for** your infrastructure. All Terraform commands should now work.

If you ever **set** or change modules or backend configuration **for** Terraform, rerun this **command** to reinitialize your working directory. If you forget, other commands will detect it and remind you to **do** so **if** necessary.

```
→ tf-vault-basic-demo git:(main) x terraform fmt
→ tf-vault-basic-demo git:(main) x terraform validate
Success! The configuration is valid.
```

```
→ tf-vault-basic-demo git:(main) x terraform plan
data.vault_auth_backends.vault-dev-server1: Reading...
data.vault_auth_backends.vault-master-server1: Reading...
data.vault_auth_backends.vault-dev-server1: Read complete
after 0s [id=default]
data.vault_auth_backends.vault-master-server1: Read complete
after 0s [id=default]
```

Changes to Outputs:

```

+ vault_dev_server1_auth_lists = [
  + {
    + accessors = [
      + "auth_token_dea3ldec",
    ]
    + id        = "default"
    + namespace = null
    + paths     = [
      + "token",
    ]
    + type      = ""
  },
]
+ vault_master_server1_auth_lists = [
  + {
    + accessors = [
      + "auth_token_dea3ldec",
    ]
    + id        = "default"
    + namespace = null
    + paths     = [
      + "token",
    ]
    + type      = ""
  },
]

```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Note: You didn't use the `-out` option to save this plan, so

Terraform can't guarantee to take exactly these actions if you run

"terraform apply" now.

```
→ tf-vault-basic-demo git:(main) x terraform apply
data.vault_auth_backends.vault-master-server1: Reading...
data.vault_auth_backends.vault-dev-server1: Reading...
data.vault_auth_backends.vault-master-server1: Read complete after 0s [id=default]
data.vault_auth_backends.vault-dev-server1: Read complete after 0s [id=default]
```

Changes to Outputs:

```
+ vault_dev_server1_auth_lists = [
  + {
    + accessors = [
      + "auth_token_dea3ldec",
    ]
    + id        = "default"
    + namespace = null
    + paths     = [
      + "token",
    ]
    + type      = ""
  },
]
+ vault_master_server1_auth_lists = [
  + {
    + accessors = [
      + "auth_token_dea3ldec",
    ]
    + id        = "default"
    + namespace = null
    + paths     = [
      + "token",
    ]
    + type      = ""
  },
]
```

```
    },  
  ]  
}
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
vault_dev_server1_auth_lists = [  
  {  
    "accessors" = tolist([  
      "auth_token_dea3ldec",  
    ])  
    "id" = "default"  
    "namespace" = toString(null)  
    "paths" = tolist([  
      "token",  
    ])  
    "type" = ""  
  },  
]  
vault_master_server1_auth_lists = [  
  {  
    "accessors" = tolist([  
      "auth_token_dea3ldec",  
    ])  
    "id" = "default"  
    "namespace" = toString(null)  
    "paths" = tolist([  
      "token",  
    ])  
    "type" = ""  
  },  
]
```

```
    ])  
    "id" = "default"  
    "namespace" = toString(null)  
    "paths" = tolist([  
      "token",  
    ])  
    "type" = ""  
  },  
]  
→ tf-vault-basic-demo git:(main) x
```